書籍管理システム（Global Books Vue3-Django）- スターターキット

これは、Vue.js 3とDjango REST Frameworkを使用して書籍管理システムを構築するためのスターターキットです。 基本的なプロジェクト構成や設定は完了していますが、主要な機能は未実装です。 以下の手順に従って、開発を進めてください。

機能要件（実装目標）

- 書籍一覧表示（画像付き）
- 書籍詳細表示
- 新規書籍登録
- 書籍編集
- 書籍削除

技術スタック

- **Frontend**: Vue.js 3, Vite, Bootstrap 5
- **Backend**: Django 5, Django REST Framework
- **Database**: SQLite3

セットアップ & 開発手順

前提条件

- Python 3.10+（演習環境は3.13）
- Node.js 20.19以上の20系、または22.12以上（今回のVite 7に合わせて変更）
- SQLite3コマンドラインツール（`sqlite3` コマンドを実行できること）

追記： 演習で使うPythonパッケージを、最初に1回インストールしてください。

```
python -m pip install "Django>=5.2.10,<5.3" "djangorestframework==3.16.1" "django-cors-headers==4.9.0"
```

1. バックエンド (Django) のセットアップ

バックエンド用のDjangoディレクトリと、DB作成用SQLだけ準備してあります。 以下の手順に従いセットアップを行ってください。

1. **ディレクトリ移動**

   このREADMEのあるフォルダから実行します。

```
cd backend-django
```

2. **Djangoプロジェクトの作成**

   Djangoプロジェクトを作成します。

```
django-admin startproject config .
```

3. **アプリケーションの作成**

   書籍管理用のアプリ `books` を作成します。

```
python manage.py startapp books
```

4. **ディレクトリ構成 (Backend)**

   以下のディレクトリ構成になっていることを確認してください。 （実行結果は一部省略して表示しています）

```
tree /f ./
```

```
backend-django
│  initial_data.sql
│  manage.py
│
├─books
│  │  models.py
│  │  views.py
│  │
│  └─migrations
│
└─config
   │  settings.py
   └─ urls.py
```

5. **設定の追加**

`config/settings.py` に必要な情報を追加してください。

- `INSTALLED_APPS` にアプリを追加

```
'rest_framework',  # 追加: REST API用
'corsheaders',     # 追加: CORS用
'books',           # 新規: 書籍管理アプリ
```

- `MIDDLEWARE` にCORSの設定を先頭に追加

```
'corsheaders.middleware.CorsMiddleware',
```

- `CORS_ALLOWED_ORIGINS` を定義して、オリジンサーバを追加

```
CORS_ALLOWED_ORIGINS = [
    'http://localhost:5173',
    'http://localhost:5174',
    'http://localhost:5175',
]
```

6. **モデルの作成**

テーブル定義を基にモデルを作成してください。以下に注意して作成します。

- ID列は自動生成とし、フィールド定義は不要です。
- nullableとはNullを許可する設定で、`null=True, blank=True` の設定で実現します。
- テーブル名は `books` とします。アプリケーション名がつかないように注意してください。モデル内の `class Meta` に `db_table = 'books'` を指定します。（Djangoのデフォルトは「アプリ名_モデル名」で生成）

追記（追加機能用の列）： `Book` モデル内に、次の1行も追加してください。後から列を追加する作業を避けるため、最初から用意します。基本機能では使わず、初期値0のまま保持します。

```
like_count = models.PositiveIntegerField(default=0)
```

7. **データベースのマイグレーション**

   モデルを作成した後に実行します。

```
python manage.py makemigrations
python manage.py migrate
```

8. **初期データの投入（SQL）**

   `backend-django` フォルダに `initial_data.sql` が用意されています。 SQLite3コマンドラインツールでデータを投入してください。 データ投入後、テーブルのレコードが表示されるため、確認してください。

```
sqlite3 db.sqlite3
.read initial_data.sql
.quit
```

   追記： 空の書籍テーブルに初回だけ投入します。`.read` と `.quit` はSQLiteのプロンプト内で実行し、10件の表示を確認してください。

9. **サーバー起動**

```
python manage.py runserver
```

   標準ポート8000で起動します。

2. フロントエンド（Vue）のセットアップ

フロントエンドは、必要な画面やコンポーネントのファイル（枠組み）のみ用意されています。

1. **ディレクトリ移動とインストール**

   別のターミナルを開き、このREADMEのあるフォルダから実行します。

```
cd frontend
npm install
```

2. **開発サーバー起動**

```
npm run dev
```

3. **ブラウザで確認**

   アクセス: http://localhost:5173

   画面に表示される指示（コメント）に従って、実装を進めてください。

補足

・ APIの仕様や画面設計については、別途配布される設計書を確認してください。

追記：今回の追加機能（任意）

基本機能を完成させた後、次の3つに任意で取り組めます。

・ 並び替えボタンの追加

・ 書籍ごとのいいね

・ 画像ファイル添付

いいね用の列は手順6で用意済みです。APIの実装時には、`BookSerializer` の `class Meta` に `read_only_fields = ['like_count']` を指定し、通常の登録・編集で回数を上書きしないようにします。

画像ファイル添付に取り組む場合だけ、Pillowを追加でインストールしてください。

```
python -m pip install "Pillow>=11.1,<13"
```

画面例は同梱の基本設計書の第7章「追加機能」を確認してください。